CSE 4303
Data Structures

Topic: Stack

Sabbir Ahmed
Assistant Prof | CSE | IUT
sabbirahmed@iut-dhaka.edu

Stack

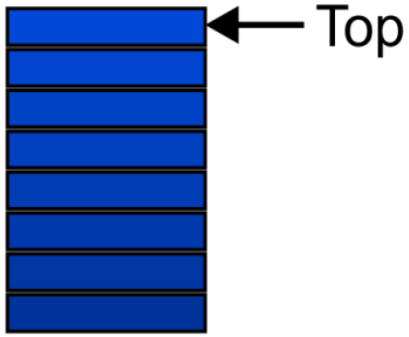
- *Last-in-First-out* (LIFO) behavior
- The last item to be inserted into a stack is the first one to be deleted from it.

Stack

- *Last-in-First-out* (LIFO) behavior
- The last item to be inserted into a stack is the first one to be deleted from it.
- For example, you have a **stack of trays** on a table. The tray at the top of the stack is the first item to be moved if you require a tray from that stack.

Stack

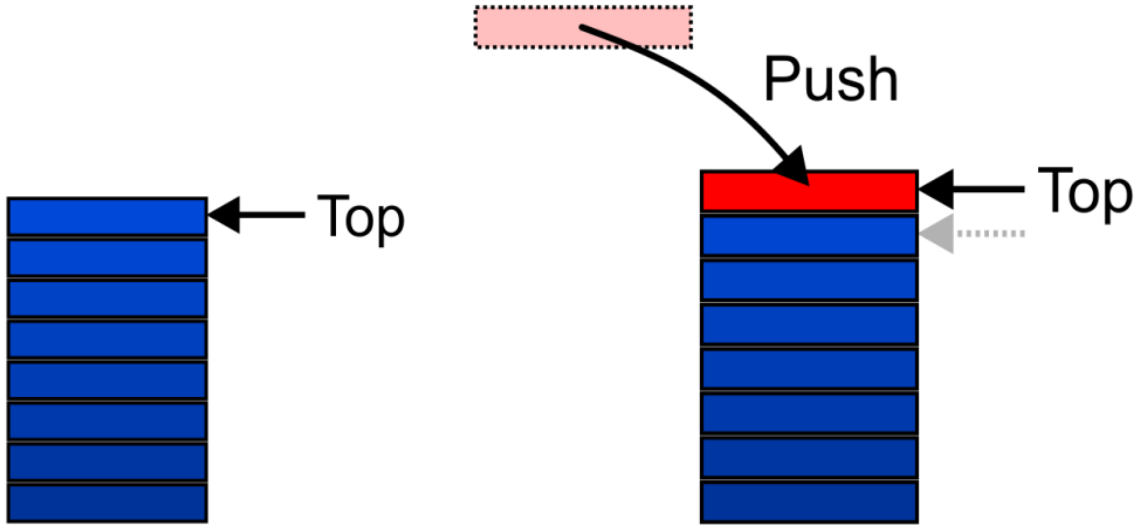
- *Last-in-First-out* (LIFO) behavior



top of the stack: most recent item ***pushed*** onto the stack

Stack

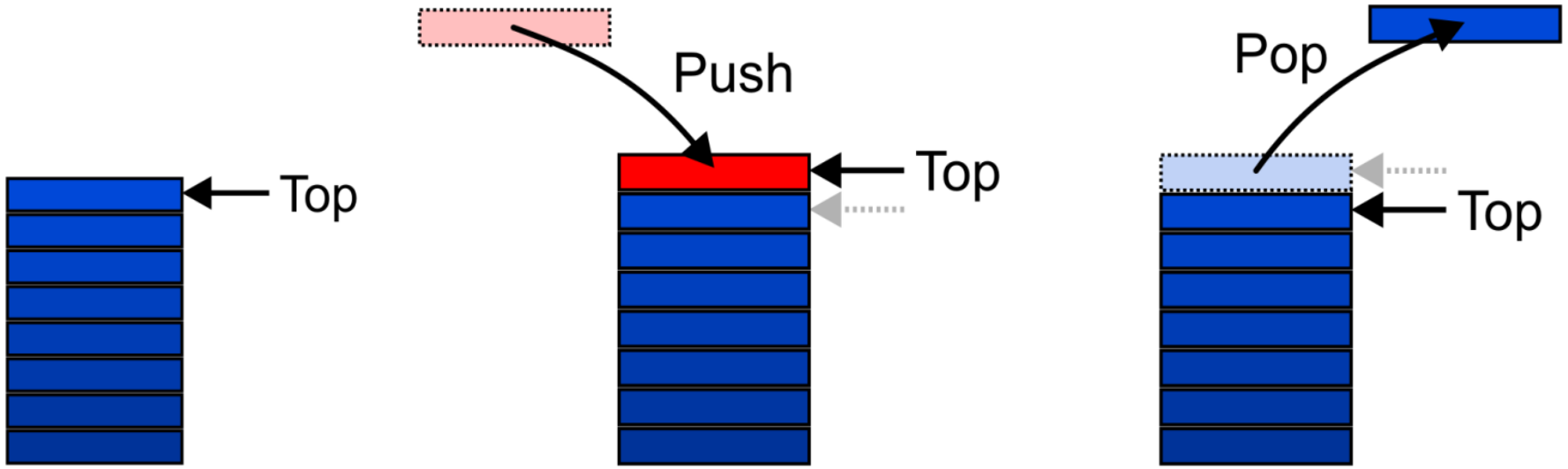
- *Last-in-First-out* (LIFO) behavior



Push: inserting an item on the ***top*** of the stack

Stack

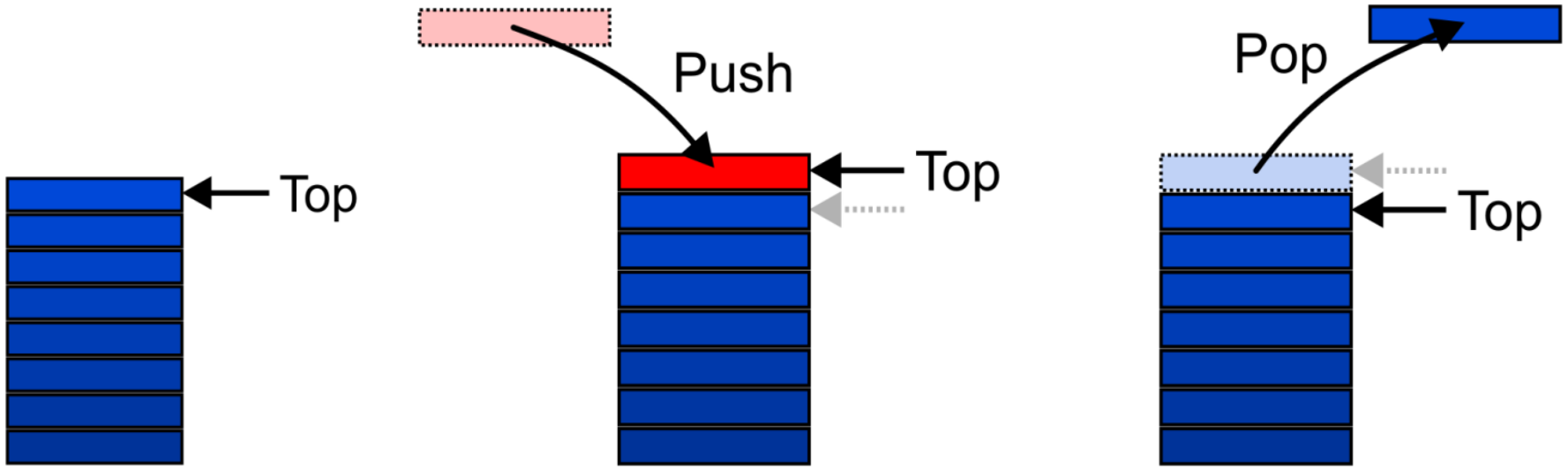
- *Last-in-First-out* (LIFO) behavior



pop: removing an item from the ***top*** of the stack

Stack

- *Last-in-First-out* (LIFO) behavior

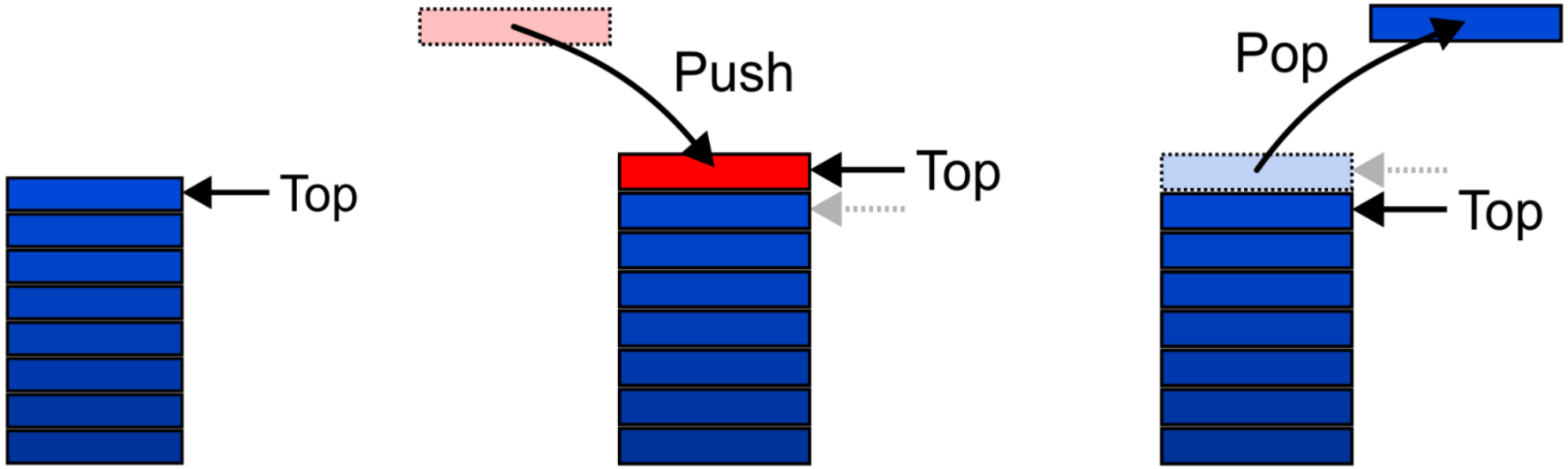


pop: removing an item from the ***top*** of the stack

Stack

- *Last-in-First-out* (LIFO) behavior

stack operations only take place at the top of stack. Not elsewhere!!!



pop: removing an item from the ***top*** of the stack

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking recursive function calls

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking recursive function calls
- Dealing with undo/redo operations

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking recursive function calls
- Dealing with undo/redo operations
- Reverse-Polish calculators

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking recursive function calls
- Dealing with undo/redo operations
- Reverse-Polish calculators
- Assembly language

Applications

- Parsing code
  - Matching parenthesis
  - XML (e.g., XHTML)
- Tracking recursive function calls
- Dealing with undo/redo operations
- Reverse-Polish calculators
- Assembly language

The stack is a very simple data structure!

Given any problem, if it is possible to use a stack, this will significantly simplify the solution

Stack Operations

`boolean isEmpty(stack);`	return true if empty

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full
`void push(stack, item);`	insert item into stack

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full
`void push(stack, item);`	insert item into stack
`void pop(stack);`	remove most recent item

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full
`void push(stack, item);`	insert item into stack
`void pop(stack);`	remove most recent item
`void clear(stack);`	remove all items from stack

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full
`void push(stack, item);`	insert item into stack
`void pop(stack);`	remove most recent item
`void clear(stack);`	remove all items from stack
`item top(stack);`	retrieve most recent item

Stack Operations

`boolean isEmpty(stack);`	return true if empty
`boolean isFull(stack);`	return true if full
`void push(stack, item);`	insert item into stack
`void pop(stack);`	remove most recent item
`void clear(stack);`	remove all items from stack
`item top(stack);`	retrieve most recent item
`item topAndPop(stack);`	return & remove most recent item

Stack Operations

Time complexity?

Stack Operations

The optimal asymptotic runtime for stack operations is

O(1)

The run time should be independent of the number of objects being stored in the container

Stack Operations

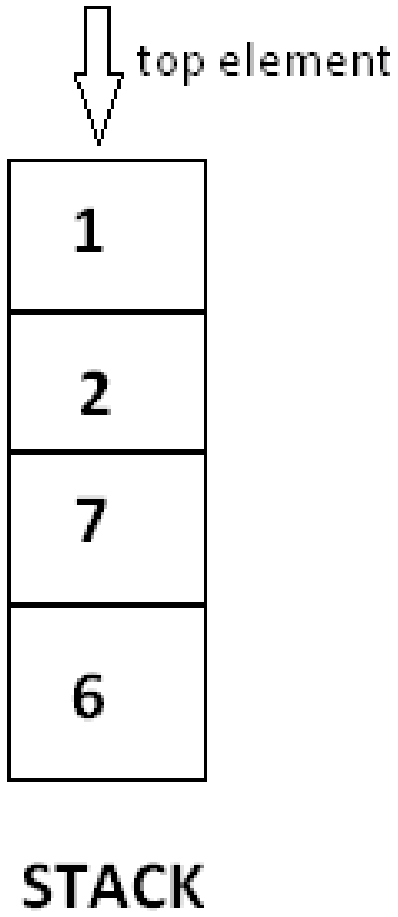
The optimal asymptotic runtime for stack operations is

O(1)

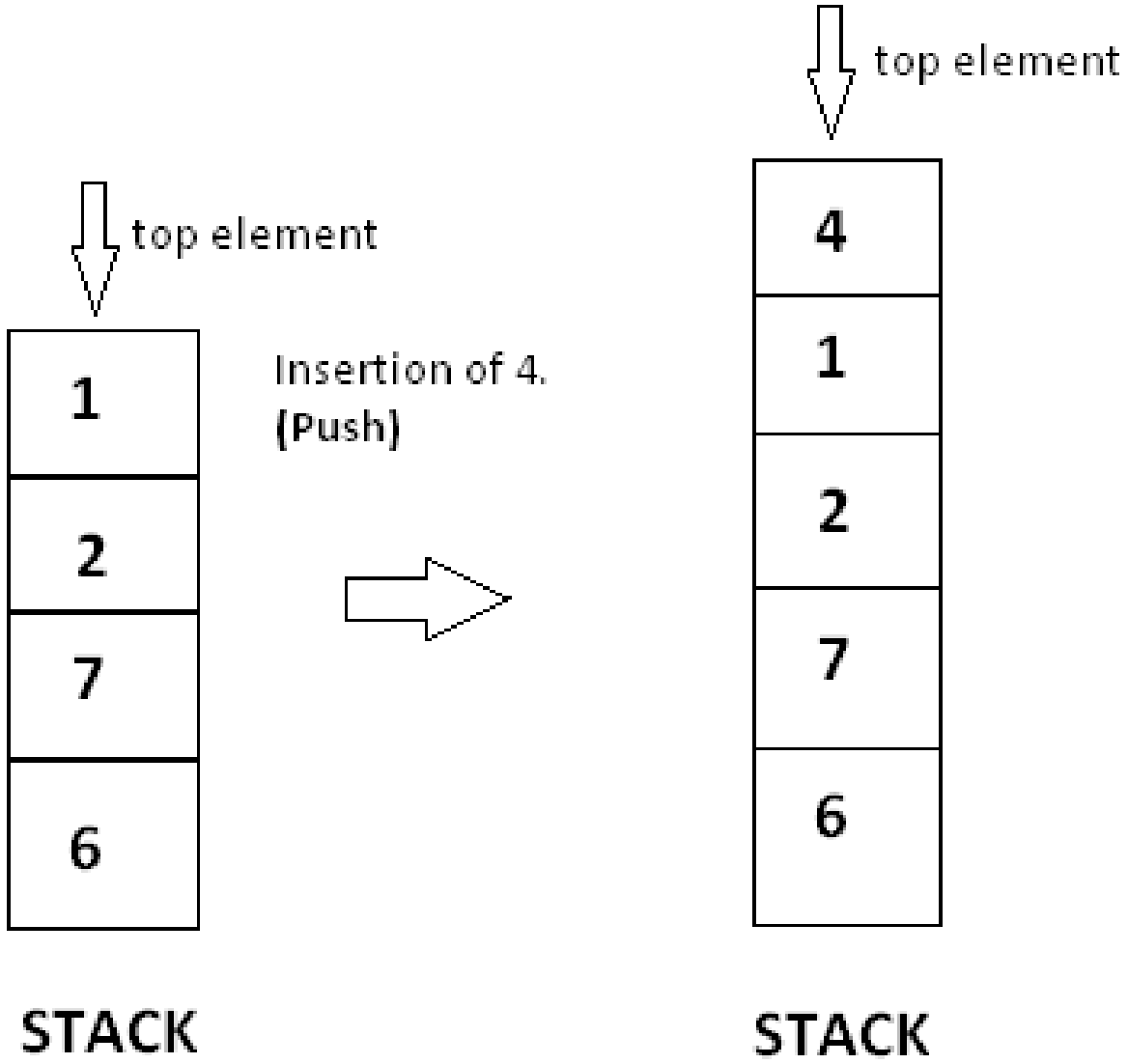
The run time should be independent of the number of objects being stored in the container

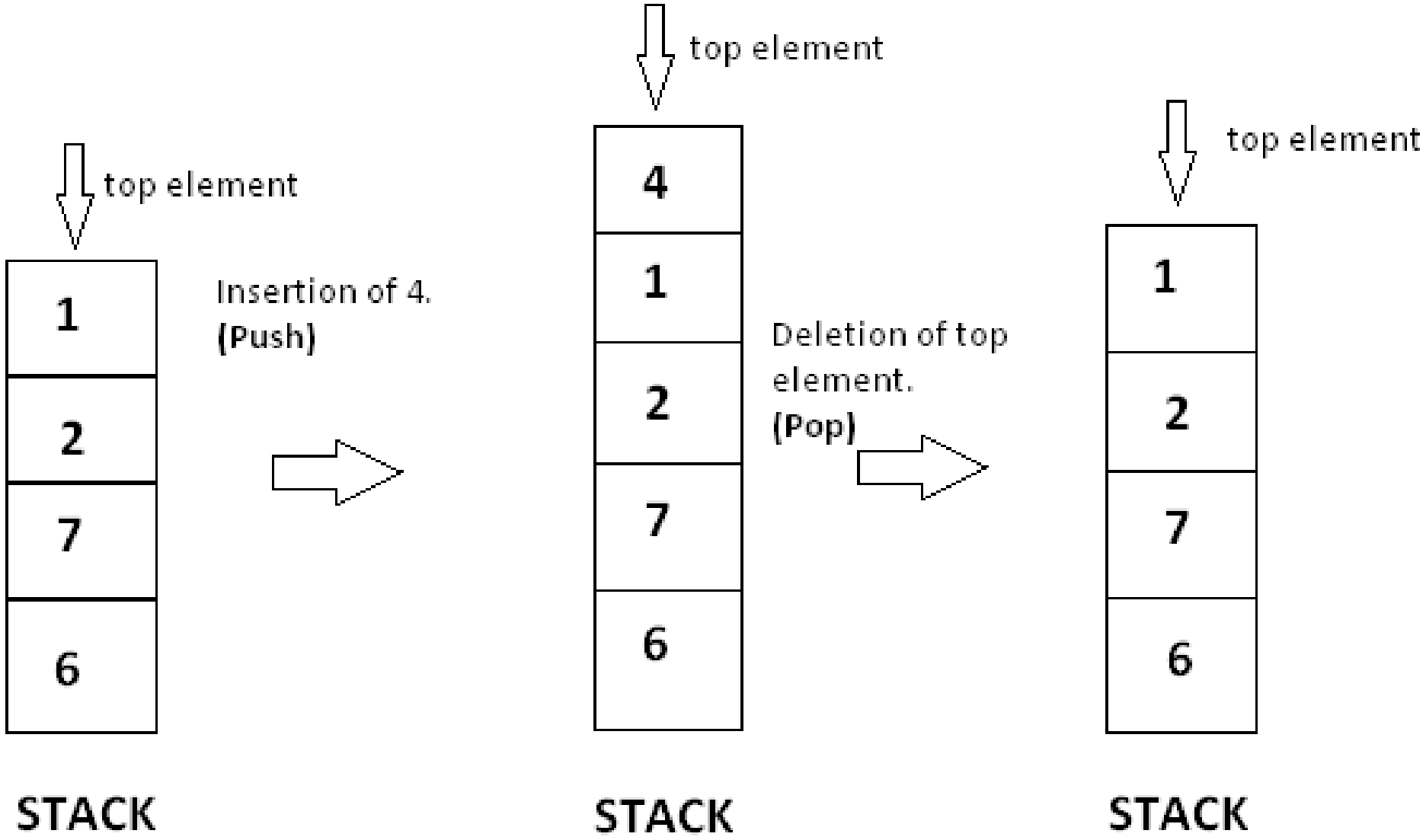
We will look at implementations based on --

- One-ended arrays (this lecture)
- Singly linked lists (after a few lectures)



STACK


top element
1
2
7
6
Insertion of 4.
(Push)
STACK
top element
4
1
2
7
6
STACK


top element
1
2
7
6
Insertion of 4.
(Push)
STACK
top element
4
1
2
7
6
Deletion of top
element.
(Pop)
STACK
top element
1
2
7
6
STACK

```
void push( int stack[ ], int x, int n) {

}
```

```
void push( int stack[ ], int x, int n) {

        top = top + 1 ;        // Incrementing top position
        stack[ top ] = x ;     // Inserting element on
                               incremented position

}
```

```
void push( int stack[ ], int x, int n) {

        top = top + 1 ;          // Incrementing top position
        stack[ top ] = x ;       // Inserting element on
                                 incremented position

}
```

What should be the initial value of top ?

```
void push( int stack[ ], int x, int n) {

        top = top + 1 ;          // Incrementing top position
        stack[ top ] = x ;       // Inserting element on
                                 incremented position

}
```

What should be the initial value of top ?
(initiate top=-1)

```
void push( int stack[ ], int x, int n) {

        top = top + 1 ;          // Incrementing top position
        stack[ top ] = x ;       // Inserting element on
                                 incremented position

}
```

What if the stack overflows?

```
void push( int stack[ ], int x, int n) {
      if ( top == n-1 ) {


            cout << “Stack is full. Overflow condition!” ;
      }
      else {
            top = top + 1 ;          // Incrementing top position
            stack[ top ] = x ;       // Inserting element on
                                        incremented position
    }
}
```

What is the Time complexity of push() ?

```
void push( int stack[ ], int x, int n) {
      if ( top == n-1 ) {


            cout << “Stack is full. Overflow condition!” ;
      }
      else {
            top = top + 1 ;          // Incrementing top position
            stack[ top ] = x ;       // Inserting element on
                                        incremented position
    }
}
```

```
void push( int stack[ ], int x, int n) {
      if ( top == n-1 ) {


            cout << “Stack is full. Overflow condition!” ;
      }
      else {
            top = top + 1 ;          // Incrementing top position
            stack[ top] = x ;        // Inserting element on
                                     incremented position
   }
}
```

What is the Time complexity of push() ?

O(1)

```
void pop (int stack[ ] ,int n ) {

}
```

```
void pop (int stack[ ] ,int n ) {

        top = top - 1;

}
```

```
void pop (int stack[ ] ,int n ) {

        top = top - 1; // Decrementing top's position
                       will detach the last element
                           from the stack

}
```

```
void pop (int stack[ ] ,int n ) {

        top = top - 1; // Decrementing top's position
                       will detach the last element
                           from the stack

}
```

What if the stack is empty when pop() is called?

```
void pop (int stack[ ] ,int n ) {
     if( isEmpty() ){
          cout << “Stack is empty. Underflow!” << endl;
     }
     else {
          top = top - 1; // Decrementing top’s position
                              will detach the last element
                                   from the stack
     }
}
```

What is the Time complexity of pop() ?

```
void pop (int stack[ ] ,int n ) {
     if( isEmpty() ){
          cout << “Stack is empty. Underflow!” << endl;
     }
     else {
          top = top - 1; // Decrementing top’s position
                              will detach the last element
                                   from the stack
     }
}
```

```
void pop (int stack[ ] ,int n ) {
     if( isEmpty() ){
          cout << “Stack is empty. Underflow!” << endl;
     }
     else {
          top = top - 1; // Decrementing top’s position
                              will detach the last element
                                   from the stack
     }
}
```

What is the Time complexity of pop() ?

O(1)

```
bool isEmpty (int stack[ ])
    {




    }
```

```
bool isEmpty (int stack[ ])
   {
       if ( top == -1 )  //Stack is empty
              return true ;
       else
              return false;
   }
```

```
int topElement (int stack[ ])
    {


    }
```

```
int topElement (int stack[ ])
   {
        if( !isEmpty() )
             return stack[ top ];
   }
```

```
int size ( )
    {

    }
```

```
int size ( )
    {
        return top + 1;
    }
```



	Front/1st	Back/n^{th}
Find top	O(?)	O(?)
Insert	O(?)	O(?)
Erase	O(?)	O(?)



	Front/1st	Back/n^{th}
Find top	O(1)	O(1)
Insert	O(?)	O(1)
Erase	O(?)	O(1)



	Front/1st	Back/n^{th}
Find top	O(1)	O(1)
Insert	O(N)	O(1)
Erase	O(?)	O(1)

Shift items to the right side to create a space for the new item



	Front/1st	Back/n^{th}
Find top	O(1)	O(1)
Insert	O(N)	O(1)
Erase	O(N)	O(1)

pop.

Shift items to the left side to fill the erased item's space

Applications: Parsing

- Most parsing approaches use stacks

- Examples include:
  - Matching tags in XHTML
  - In C++, matching
    - Parentheses (…)
    - Brackets […]
    - braces { … }

HTML parsing

- HTML is made of nested
  - opening tags, e.g., `<some_identifier>`, and
  - matching closing tags, e.g., `</some_identifier>`

HTML parsing

- HTML is made of nested
  - opening tags, e.g., <some_identifier>, and
  - matching closing tags, e.g., </some_identifier>

```
<html>
   <head><title> Hello </title></head>
   <body><p> This appears in the
   <i> browser </i>.</p></body>
</html>
```

HTML parsing

- Nesting indicates that any *closing tag must match the most recent opening tag*

- Strategy for parsing HTML:
  - ???
  - ???
  - ???

HTML parsing

- Nesting indicates that any *closing tag must match the most recent opening tag*

- Strategy for parsing HTML:
  - read through the HTML linearly
  - ???
  - ???

HTML parsing

- Nesting indicates that any *closing tag must match the most recent opening tag*

- Strategy for parsing HTML:
  - read through the HTML linearly
  - place the opening tags in a stack
  - ???

HTML parsing

- Nesting indicates that any *closing tag must match the most recent opening tag*

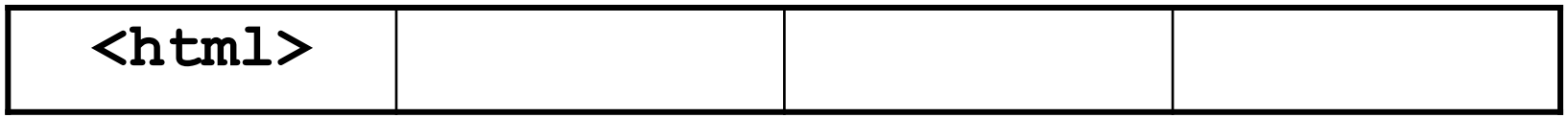
- Strategy for parsing HTML:
  - read through the HTML linearly
  - place the opening tags in a stack
  - when a closing tag is encountered, check ???

HTML parsing

- Nesting indicates that any *closing tag must match the most recent opening tag*

- Strategy for parsing HTML:
  - read through the HTML linearly
  - place the opening tags in a stack
  - when a closing tag is encountered, check whether it matches with top of the stack

```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



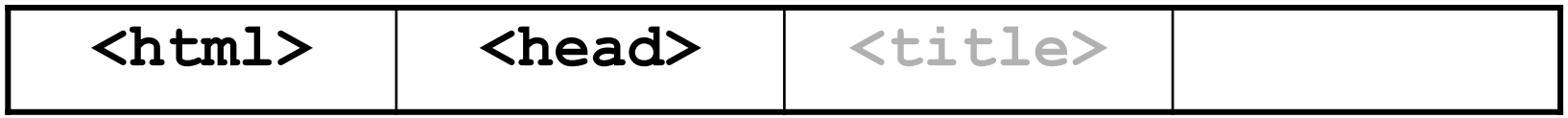
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```

<html>	<head>		

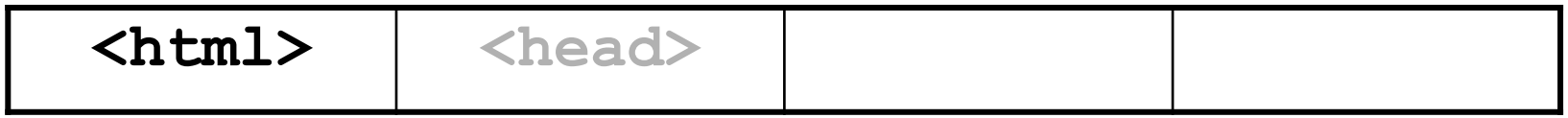
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```

<html>	<head>	<title>	

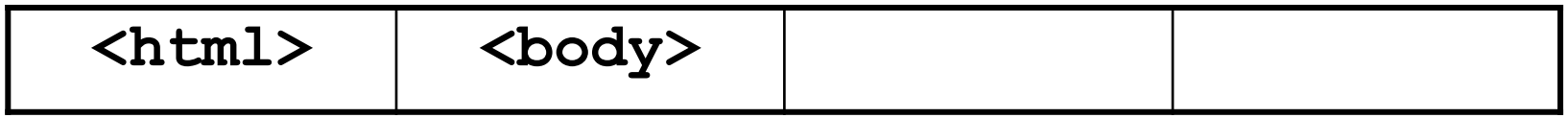
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```

<html>	<body>	<p>	

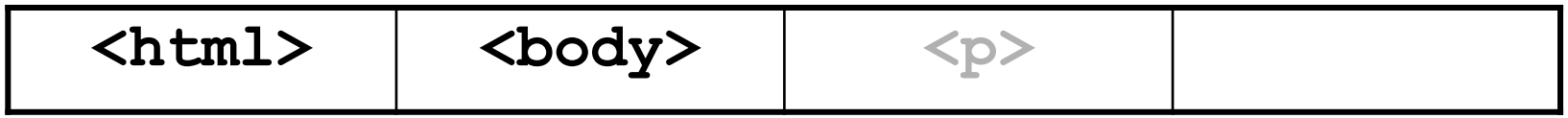
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```

<html>	<body>	<p>	<i>

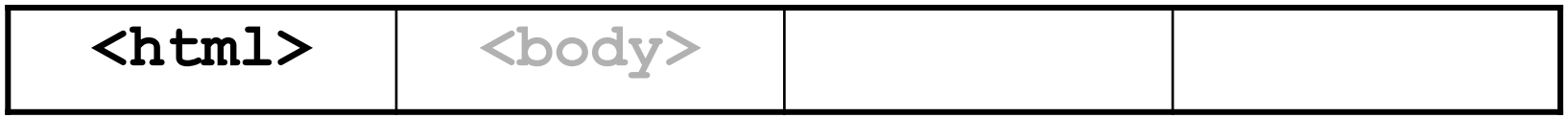
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```

<html>	<body>	<p>	<i>

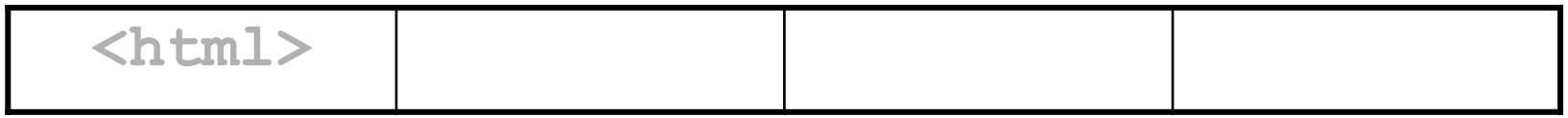
```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



```
<html>
<head><title>Hello</title></head>
<body><p>This appears in the
<i>browser</i>.</p></body>
</html>
```



- We are finished parsing, and the stack is empty

- Possible errors:
  - ???
  - ???
  - ???

- We are finished parsing, and the stack is empty

- Possible errors:
  - The stack is not empty at the end of the document
  - ???
  - ???

- We are finished parsing, and the stack is empty

- Possible errors:
  - The stack is not empty at the end of the document
  - A closing tag that does not match the opening tag on top of the stack
  - ???

- We are finished parsing, and the stack is empty

- Possible errors:
  - The stack is not empty at the end of the document
  - A closing tag that does not match the opening tag on top of the stack
  - A closing tag when the stack is empty

Applications: Parsing C++

- Like opening and closing tags, C++ parentheses, brackets, and braces must be similarly nested:

Applications: Parsing C++

- Like opening and closing tags, C++ parentheses, brackets, and braces must be similarly nested:

```
void initialize( int *array, int n ) {
    for ( int i = 0; i < n; ++i ) {
        array[i] = 0;
    }
}
```

Applications: Parsing C++

- Like opening and closing tags, C++ parentheses, brackets, and braces must be similarly nested:

```
void initialize( int *array, int n ) {
    for ( int i = 0; i < n; ++i ) {
        array[i] = 0;
    }
}
```

Applications: Parsing C++

- Like opening and closing tags, C++ parentheses, brackets, and braces must be similarly nested:

```
void initialize( int *array, int n ) {
    for ( int i = 0; i < n; ++i ) {
        array[i] = 0;
    }
}
```

For C++, the errors are similar to that for HTML

Applications: Parsing C++

- Like opening and closing tags, C++ parentheses, brackets, and braces must be similarly nested:

```
void initialize( int *array, int n ) {
    for ( int i = 0; i < n; ++i ) {
        array[i] = 0;
    }
}
```

How do we implement this in code?

For C++, the errors are similar to that for HTML

Applications: Bracket Matching Problem

Ensures that pairs of brackets are properly matched

- An Example: `{a,(b+f[4])*3,d+f[5]}`



- Bad Examples:

```
(..)..)          // too many closing brackets

(..(..)          // too many open brackets

[..(..]..)       // mismatched brackets
```



Applications: Bracket Matching Problem

Ensures that pairs of brackets are properly matched

- An Example: `{a,(b+f[4])*3,d+f[5]}`



How do we implement this in code?

- Bad Examples:

```
(..)..)          // too many closing brackets

(..(..)          // too many open brackets

[..(..]..)       // mismatched brackets
```



Applications: Function calls

- Function calls are similar to problems presented earlier:
  - you write a function to solve a problem
  - the function may require sub-problems to be solved, hence, it may call another function
  - once a function is finished, it returns to the function which called it

Applications: Function calls

- Function calls are similar to problems presented earlier:
  - you write a function to solve a problem
  - the function may require sub-problems to be solved, hence, it may call another function
  - once a function is finished, it returns to the function which called it

- You will notice that when a function returns, execution and the return value are ***passed back to the last function which was called***

Applications: Function calls

- Function calls are similar to problems presented earlier:
  - you write a function to solve a problem
  - the function may require sub-problems to be solved, hence, it may call another function
  - once a function is finished, it returns to the function which called it

- You will notice that when a function returns, execution and the return value are ***passed back to the last function which was called***
- This is again, the ***last-in-first-out property***

Applications: Reverse-Polish Notation

- Normally, mathematics is written using what we call '**in-fix notation**':

$$(3 + 4) \times 5 - 6$$

- The operator is placed between two operands

Applications: Reverse-Polish Notation

- Normally, mathematics is written using what we call '**in-fix notation**':

$$(3 + 4) \times 5 - 6$$

- The operator is placed between two operands

- One weakness: parentheses are required

$$(3 + 4) \times 5 - 6 = 29$$
$$3 + 4 \times 5 - 6 = 17$$
$$3 + 4 \times (5 - 6) = -1$$
$$(3 + 4) \times (5 - 6) = -7$$

Applications: Reverse-Polish Notation

- Alternatively, we can place the operands first, followed by the operator:

  (3 + 4) × 5 – 6

  3 4 + 5 × 6 –

- Alternatively, we can place the operands first, followed by the operator:

  (3 + 4) × 5 – 6

  3 4 + 5 × 6 –

- Parsing reads left-to-right and performs any operation on the last two operands:

  3 4 + 5 × 6 –

  7 5 × 6 –

  35 6 –

  29

3 4 5 × + 6 − = ???

3 4 5 × + 6 −

3 20 + 6 −

23 6 −

17

---

3 4 5 6 - × + = ???

3 4 5 × + 6 −

3 20 + 6 −

23 6 −

17

---

3 4 5 6 - × +

3 4 -1 × +

3 -4 +

-1

Reverse-Polish Notation

- Benefits:
  - **No ambiguity** and **no brackets** are required
  - It is the same process used by a computer to perform computations:
    - operands must be loaded into registers before operations can be performed on them

Reverse-Polish Notation

- Benefits:
  - **No ambiguity** and **no brackets** are required
  - It is the same process used by a computer to perform computations:
    - operands must be loaded into registers before operations can be performed on them
  - *Reverse-Polish can be processed using stacks!!!*

Reverse-Polish Notation

- Benefits:
  - **No ambiguity** and **no brackets** are required
  - It is the same process used by a computer to perform computations:
    - operands must be loaded into registers before operations can be performed on them
  - *Reverse-Polish can be processed using stacks!!!*

How do we implement this in code?

Reverse-Polish Notation

- The easiest way to parse Reverse-Polish notation is to use an ***operand stack***:
  - **operands** are processed by pushing them onto the stack
  - when processing an **operator**:
    - ???
    - ???
    - ???

Reverse-Polish Notation

- The easiest way to parse Reverse-Polish notation is to use an ***operand stack***:
  - **operands** are processed by pushing them onto the stack
  - when processing an **operator**:
    - pop the last two items off the operand stack,
    - ???
    - ???

Reverse-Polish Notation

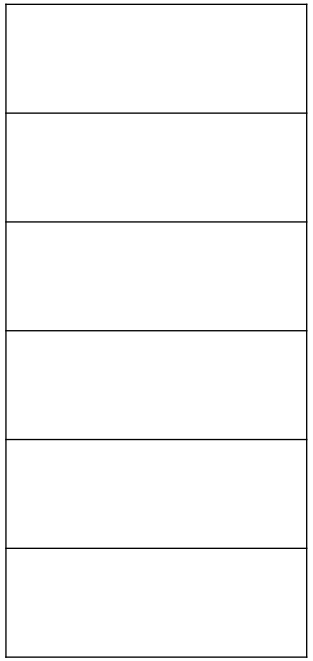
- The easiest way to parse Reverse-Polish notation is to use an ***operand stack***:
  - **operands** are processed by pushing them onto the stack
  - when processing an **operator**:
    - pop the last two items off the operand stack,
    - perform the operation, and
    - ???

Reverse-Polish Notation

- The easiest way to parse Reverse-Polish notation is to use an ***operand stack***:
  - **operands** are processed by pushing them onto the stack
  - when processing an **operator**:
    - pop the last two items off the operand stack,
    - perform the operation, and
    - push the result back onto the stack

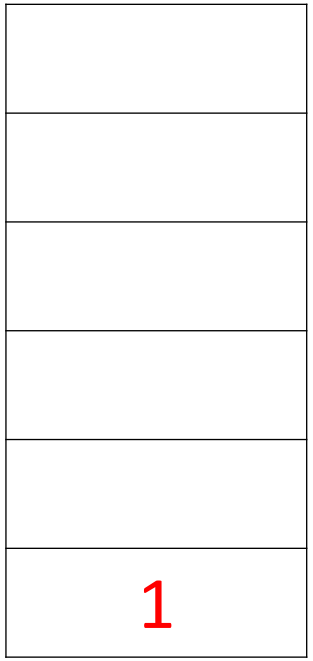
Evaluate the following reverse-Polish expression using a stack:

$$1\ \ 2\ \ 3\ +\ 4\ \ 5\ \ 6\ \times\ -\ 7\ \times\ +\ -\ 8\ \ 9\ \times\ +$$



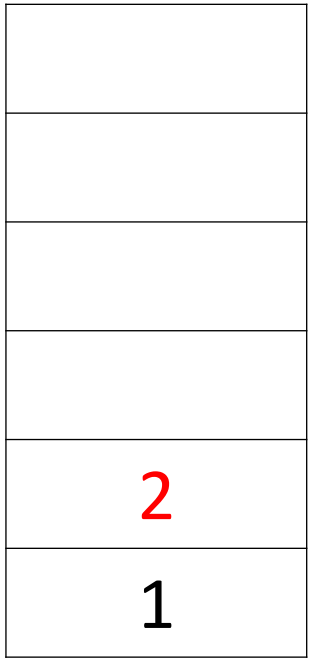
Push 1 onto the stack

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



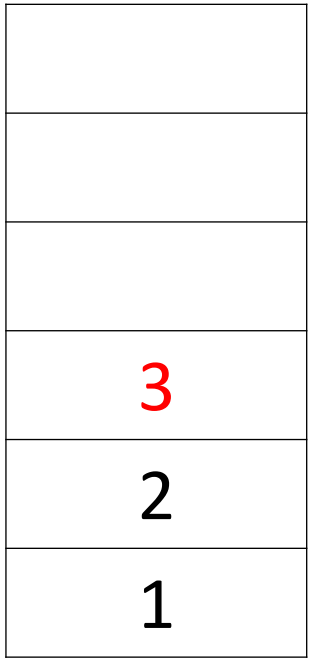
Push 1 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



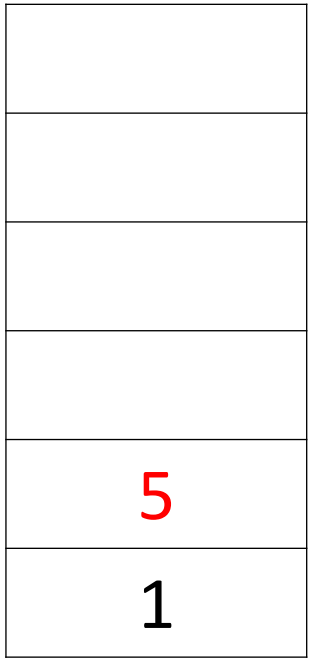
Push 3 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



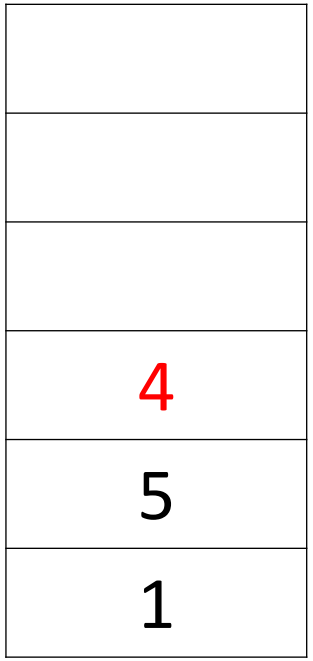
Pop two items (3,2) and push 2 + 3 = 5

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



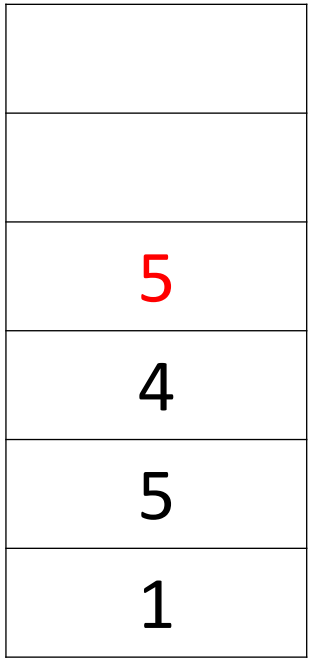
Push 4 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



Push 5 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



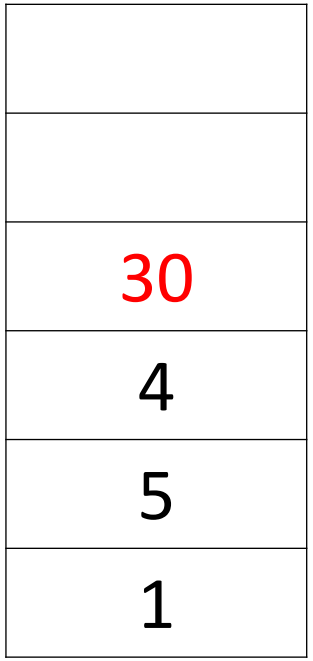
Push 6 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +

6
5
4
5
1

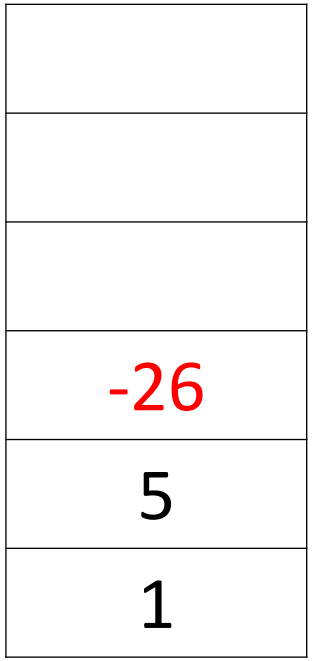
Pop 6 and 5 and push 5 × 6 = 30

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



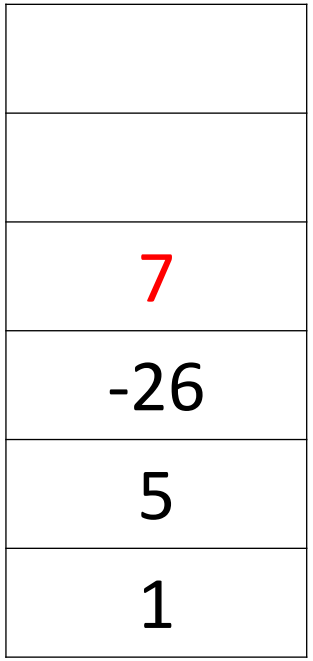
Pop 30 and 4 and push 4 – 30 = –26

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



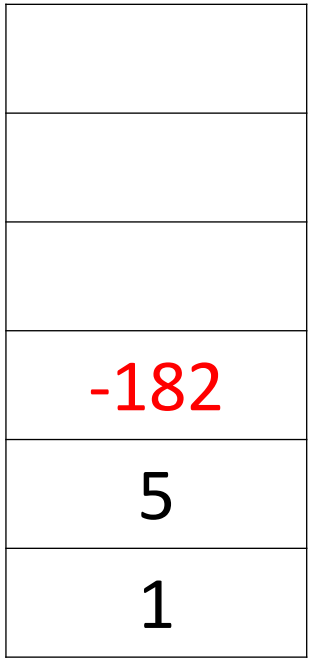
Push 7 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



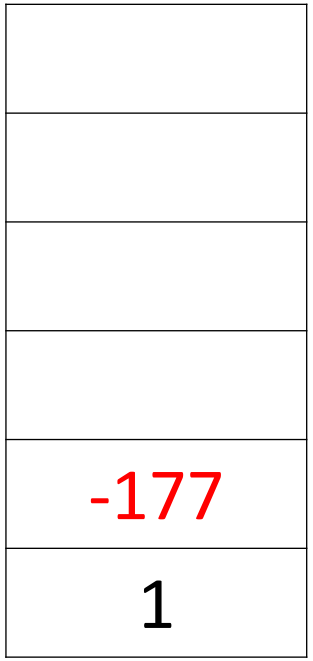
Pop 7 and –26 and push –26 × 7 = –182

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



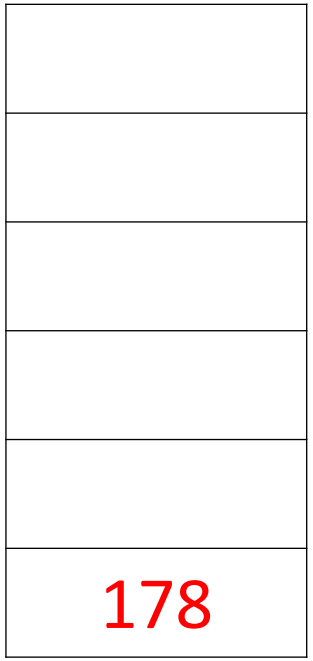
Pop –182 and 5 and push –182 + 5 = –177

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



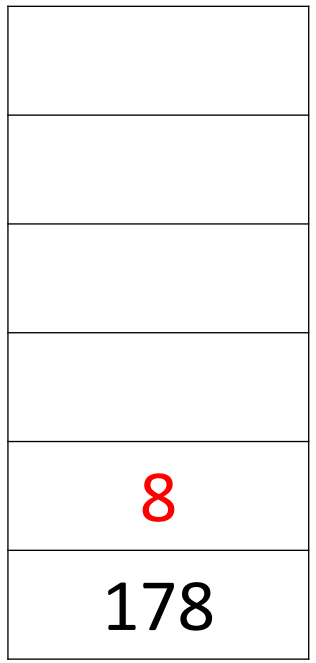
Pop –177 and 1 and push 1 – (–177) = 178

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



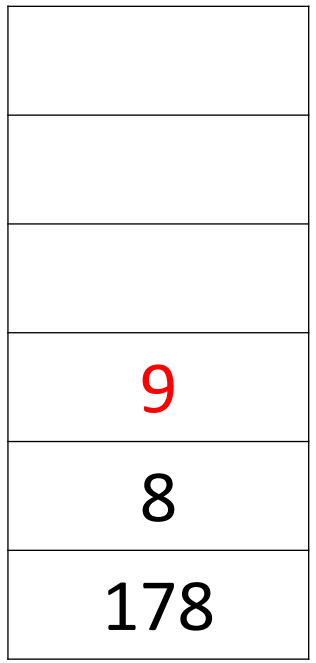
Push 8 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



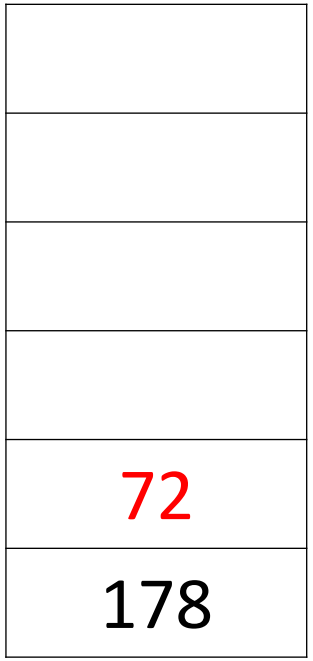
Push 1 onto the stack

1 2 3 + 4 5 6 × − 7 × + − 8 9 × +



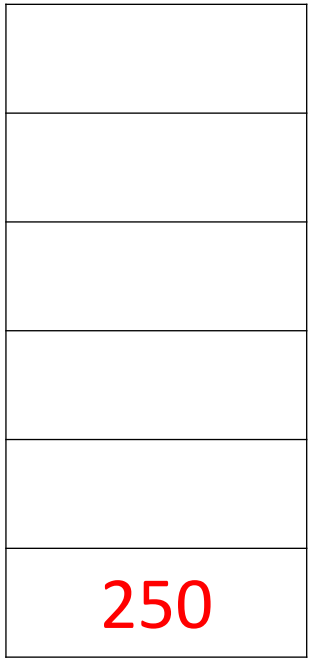
Pop 9 and 8 and push 8 × 9 = 72

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Pop 72 and 178 and push 178 + 72 = 250

1 2 3 + 4 5 6 × – 7 × + – 8 9 × +



Reverse-Polish Notation

Thus 1 2 3 + 4 5 6 × − 7 × + − 8 9 × +

evaluates to the value on the top: 250

The equivalent in-fix notation is

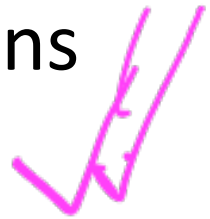
((1 − ((2 + 3) + ((4 − (5 × 6)) × 7))) + (8 × 9))

We reduce the parentheses using order-of-operations:

1 − (2 + 3 + (4 − 5 × 6) × 7) + 8 × 9

Applications

Transforming Infix Expressions into Postfix Expressions (self-study)



Standard Template Library

```
#include <iostream>
#include <stack>
using namespace std;

int main() {
        stack <int> myStack;
        myStack.push( 13 );
        myStack.push( 42 );

        cout << "Top: " << myStack.top() << endl;
        myStack.pop();    // no return value

        cout << "Top: " << myStack.top() << endl;
        cout << "Size: " << myStack.size() << endl;

        return 0;
}
```

References

- Chapter 3, ‘Data Structures and Algorithm Analysis in C++’, Mark Allen Weiss, Fourth Edition
- “Basics of Stacks Tutorials & Notes | Data Structures | HackerEarth,” HackerEarth, Apr. 26, 2016. https://www.hackerearth.com/practice/data-structures/stacks/basics-of-stacks/tutorial/

Thank you